

BRAC University
Department of Electrical and Electronic Engineering
EEE/ECE-103: Computer Programming
THMD
Final Preparation — Practice Set

Part A Structures

A1. Student registration — department code from the middle digits [12]

CORE · *struct, array of structs, digit split*

Develop a structure named **Student** that stores the **name**, the **registration number** (8-digit integer) and the **CGPA** of a student.

In a registration number the first two digits are the admission year, the next three digits are the department code, and the last three digits are the roll serial. For example 22101045 means admitted in 2022, department 101, serial 045.

Write a C program to:

- Store the records below using an array of structures.
- Take a department code *D* as input.
- Print the name and CGPA of every student of department *D*.
- Print how many of those students were admitted before 2023.

GIVEN

```
["Nabila Haque",    22101045, 3.71]
["Tanvir Ahmed",   23102187, 3.25]
["Sadia Islam",    21101302, 3.88]
["Rakib Hasan",    23103211, 2.95]
["Mahin Chowdhury", 22101150, 3.40]
```

SAMPLE RUN

```
Enter department code: 101
Nabila Haque      CGPA: 3.71 (admitted in 2022)
Sadia Islam      CGPA: 3.88 (admitted in 2021)
Mahin Chowdhury   CGPA: 3.40 (admitted in 2022)
Students found: 3
Admitted before 2023: 3
```

A2. Medicine batch — expiry from the year and month digits [12]

STRETCH · *struct, digit split, date arithmetic*

Develop a structure named **Medicine** that stores the **name**, the **batch number** (6-digit integer) and the **unit price**.

In a batch number the first two digits are the manufacturing year, the next two digits are the manufacturing month and the last two digits are a serial. For example 240718 means the batch was made in July 2024. Every medicine expires **24 months** after manufacture.

Write a C program to:

- Store the records below using an array of structures.
- Take the current year and the current month as input (for September 2026, enter 26 9).
- Print the name, manufacturing date and age in months of every *expired* medicine.
- Print how many have expired and the total price of the expired stock.

GIVEN

```
["Napa Extra", 240718, 2.50]
["Seclo 20", 230512, 8.00]
["Monas 10", 250926, 18.00]
["Fexo 120", 221105, 9.50]
["Maxpro 20", 241203, 7.00]
```

SAMPLE RUN

Enter current year and month: 26 9

Expired medicines:

```
Napa Extra   made 2024-07   age 26 months   price 2.50
Seclo 20     made 2023-05   age 40 months   price 8.00
Fexo 120     made 2022-11   age 46 months   price 9.50
```

Total expired: 3

Value lost : 20.00 Taka

A3. Employee code — bonus from the last digit

[12]

CORE · *struct, last digit, first two digits*

Develop a structure named **Employee** that stores the **name**, the **employee code** (6-digit integer) and the **monthly salary**.

The first two digits of the employee code are the joining year and the **last digit** is the grade of the employee (1 to 5). The yearly bonus is *grade* × 5% of the salary — so a grade 3 employee gets 15% of their salary.

Write a C program to:

- Store the records below using an array of structures.
- Take a year *Y* as input (for 2020, enter 20).
- For every employee who joined *before* year *Y*, print the name, joining year, grade and bonus.
- Print the name of the employee with the highest bonus among them.

GIVEN

```
["Imran Kabir", 190453, 45000]
["Farhana Yasmin", 210781, 38000]
["Shuvo Das", 170294, 62000]
["Nusrat Jahan", 220615, 41000]
```

SAMPLE RUN

```
Enter year Y: 20
Imran Kabir      joined 2019  grade 3  bonus 6750.00
Shuvo Das        joined 2017  grade 4  bonus 12400.00

Highest bonus: Shuvo Das (12400.00 Taka)
```

A4. Bank account — validating a check digit**[12]**STRETCH · *struct, digit sum, validation*

Develop a structure named **Account** that stores the **holder's name**, the **account number** (8-digit integer) and the **balance**.

The last digit of an account number is a **check digit**. An account number is valid when the check digit equals the sum of the first seven digits, modulo 10. For example 12345678 is valid because $1+2+3+4+5+6+7 = 28$ and $28 \% 10 = 8$.

Write a C program to:

- Store the records below using an array of structures.
- Check every account and print the invalid ones, showing the expected and the actual check digit.
- Print how many accounts are invalid and the total balance held in the valid accounts.

GIVEN

```
["Rafiul Islam",    12345678, 52000.00]
["Sumaiya Akter",   20439183, 78500.50]
["Jahid Hasan",     50607086, 13400.75]
["Mitu Rani",       99887764, 96000.00]
["Arman Chowdhury", 31415920, 45250.25]
```

SAMPLE RUN

```
Invalid accounts:
Sumaiya Akter      20439183  (expected check digit 7, found 3)
Arman Chowdhury    31415920  (expected check digit 5, found 0)

Invalid count : 2
Valid balance : 161400.75 Taka
```

Part B Dynamic Memory

B1. Taxi fare for several rides [10]

CORE · *malloc, slab rates, function*

Write a C program to calculate the fare of several taxi rides.

Fare rules

- First 5 km → 60 Taka flat
- Next 10 km (6–15 km) → 22 Taka per km
- Above 15 km → 18 Taka per km
- A fixed booking fee of 25 Taka is added to every ride

Program requirements

- (Optional) Write a function `calculateFare(km)` that returns the fare of one ride.
- In `main()`: take the number of rides N , **dynamically allocate** memory for N distances, read them with a loop, print the fare of each ride and the total collection.

SAMPLE RUN

```
Number of rides: 4
Distance of ride 1 (km): 3
Distance of ride 2 (km): 12
Distance of ride 3 (km): 20
Distance of ride 4 (km): 5

Ride 1:   3 km -> 85.00 Taka
Ride 2:  12 km -> 239.00 Taka
Ride 3:  20 km -> 395.00 Taka
Ride 4:   5 km -> 85.00 Taka
Total collection: 804.00 Taka
```

B2. Yearly income tax [10]

CORE · *calloc, slab rates, counting*

Write a C program to calculate the yearly income tax of several taxpayers.

Tax slabs

- First 350,000 Taka → no tax
- Next 100,000 Taka → 5%
- Next 300,000 Taka → 10%
- Remaining income → 15%

Program requirements

- (Optional) Write a function `calculateTax(income)` for one person.

- b) In `main()`: take the number of taxpayers N , dynamically allocate memory for N incomes using `calloc`, read them, print each person's tax, then print how many pay no tax at all and the total tax collected.

SAMPLE RUN

```
Number of taxpayers: 4
Yearly income of person 1: 300000
Yearly income of person 2: 500000
Yearly income of person 3: 900000
Yearly income of person 4: 350000

Person 1: income 300000.00 -> tax 0.00
Person 2: income 500000.00 -> tax 10000.00
Person 3: income 900000.00 -> tax 57500.00
Person 4: income 350000.00 -> tax 0.00
Taxpayers paying zero tax: 2
Total tax collected : 67500.00 Taka
```

B3. Marks of an unknown number of students — grow with realloc [12]

STRETCH · *malloc, realloc, sentinel, average*

A teacher does not know in advance how many marks she will enter. Write a C program that:

- Allocates space for **5 marks** to begin with.
- Keeps reading marks until the user enters **-1**.
- Whenever the array becomes full, **enlarges it by 5 more places using realloc** and prints a message saying the new capacity.
- Finally prints all the marks entered, the final capacity, the average, the highest mark, and how many marks are above the average.

The **-1** itself must not be stored or counted. SAMPLE RUN

```
Enter marks (-1 to stop):
72
65
88
91
54
79
[capacity increased to 10]
83
-1

Marks entered (7): 72 65 88 91 54 79 83
Final capacity : 10
Average       : 76.00
Highest      : 91
Above average : 4
```

B4. Salaries — more employees join later**[10]**CORE · *malloc, realloc, total, average*

Write a C program to:

- a) Take the number of employees N and dynamically allocate memory for N salaries, then read them.
- b) Ask how many employees M joined later and **resize the array** to hold $N + M$ salaries, keeping the ones already entered.
- c) Read the M new salaries.
- d) Print all the salaries, the total and the average, then free the memory.

SAMPLE RUN

```
Number of employees: 3
Salary of employee 1: 25000
Salary of employee 2: 32000
Salary of employee 3: 28000
How many employees joined later? 2
Salary of employee 4: 40000
Salary of employee 5: 35000

All 5 salaries: 25000.00 32000.00 28000.00 40000.00 35000.00
Total   : 160000.00
Average : 32000.00
```

Part C Strings

C1. Count the words, even with extra spaces [6]

CORE · *word start test, no library functions*

Write C code that counts the number of words in the string below. No input is required — the string is hard coded. **No built-in string functions may be used.**

Notice the leading spaces, the double and triple spaces between words, and the trailing spaces. A program that simply counts spaces and adds one will get this wrong. GIVEN

```
char str[] = "   EEE 103   final   exam is   near   ";
```

SAMPLE RUN

```
String : "   EEE 103   final   exam is   near   "  
Words  : 6
```

C2. Count vowels, consonants, digits, spaces and symbols [6]

CORE · *character classes, no library functions*

Write C code that scans the hard-coded string below once and reports how many vowels, consonants, digits, spaces and other characters it contains. **No built-in functions.** GIVEN

```
char str[] = "BRAC University, EEE-103 Final 2026!";
```

SAMPLE RUN

```
String      : BRAC University, EEE-103 Final 2026!  
Vowels      : 10  
Consonants  : 12  
Digits      : 7  
Spaces      : 4  
Others      : 3
```

C3. Length and reverse, without strlen [6]

CORE · *manual length, reverse, no library functions*

Write C code that finds the length of the hard-coded string below and prints it backwards. **You may not use strlen or strrev.** GIVEN

```
| char str[] = "Programming";
```

SAMPLE RUN

```
String   : Programming
Length   : 11
Reversed : gnimmargorP
```

C4. Palindrome, ignoring case and spaces [6]

STRETCH · *two pointers, filtering, no library functions*

Write C code that decides whether the hard-coded string below is a palindrome, ignoring capital letters and spaces. **No built-in functions.** GIVEN

```
| char str[] = "Never Odd Or Even";
```

SAMPLE RUN

```
String   : Never Odd Or Even
Cleaned   : neveroddoeven
Result    : Palindrome
```

C5. Toggle the case of every letter [6]

CORE · *case conversion, in place, no library functions*

Write C code that changes every capital letter of the hard-coded string below to small, and every small letter to capital, leaving digits and spaces untouched. **No built-in functions.** GIVEN

```
| char str[] = "Hello World 2026";
```

SAMPLE RUN

```
Original : Hello World 2026
Toggled  : hELLO wORLD 2026
```

C6. How many times does a character appear [6]

CORE · *searching, input, no library functions*

Write C code that takes a character from the user and counts how many times it appears in the hard-coded string below. **No built-in functions.** GIVEN

```
| char str[] = "engineering education";
```

SAMPLE RUN

```
String: engineering education
Enter a character to search: e
'e' appears 4 time(s)
```

C7. Print the string without its vowels [6]

CORE · *character classes, filtering, no library functions*

Write C code that prints the hard-coded string below with every vowel removed. **No built-in functions.** GIVEN

```
| char str[] = "Computer Programming";
```

SAMPLE RUN

```
Original      : Computer Programming
Without vowels: Cmptr Prgrmmng
```

C8. Build the acronym of a name [6]

CORE · *word start test, case conversion, no library functions*

Write C code that prints the acronym of the hard-coded name below — the first letter of every word, in capitals. **No built-in functions.** GIVEN

```
| char str[] = "Bangladesh Rural Advancement Committee";
```

SAMPLE RUN

```
Name      : Bangladesh Rural Advancement Committee
Acronym   : BRAC
```

C9. Count sentences and words per sentence**[6]***STRETCH · word start test, punctuation, no library functions*

Write C code that counts the sentences and the words in the hard-coded text below, and prints the average number of words per sentence. A sentence ends at ., ! or ?. **No built-in functions.** GIVEN

```
char str[] = "C is fun. Pointers are hard! Do you agree? Practice daily.";
```

SAMPLE RUN

```
Text      : C is fun. Pointers are hard! Do you agree? Practice daily.
Sentences : 4
Words     : 11
Average   : 2.75 words per sentence
```